Reflective Journal: Lab 07 Model Evaluation

Course: ITAI-1371 – Fundamentals of Artificial Intelligence

Authors: Arinze N Okafor, Hamza Ajmal

Assignment: Lab 07 Model evaluation

1. Conceptual Breakthroughs: Moving Beyond Accuracy

Executing Module 07 Lab - Model Evaluation.ipynb shattered my assumption that raw accuracy is an effective success metric.

Our baseline logistic regression model reached an aggregate accuracy of 80.00% (144/179 correct). However, generating the confusion matrix and classification report unmasked a severe diagnostic disparity:

- Died (Class 0): 0.82 Precision, 0.86 Recall (F -score: 0.84, support: 105).
- Survived (Class 1): 0.78 Precision, 0.73 Recall (F -score: 0.76, support: 74).

The confusion matrix revealed 20 False Negatives (missed survivors) and 15 False Positives. Despite a strong 80% headline accuracy, the model missed more than a quarter (27%) of all actual survivors. This proved that high overall accuracy often conceals unacceptable classspecific failure rates.

2. Engineering Challenges & Problem-Solving

- Pipeline Syntax Failures & Data Cleaning: The starter script arrived concatenated onto a single line, causing syntax termination. I unpacked the workflow into modular blocks, mapped categorical labels (Sex to 0 and 1), and resolved deprecation warnings by explicitly reassigning median age: Python df['Age'] = df['Age'].fillna(df['Age'].median())
- Solver Convergence: To ensure numerical stability across unscaled continuous variables (Fare, Age), I configured max_iter=1000 to prevent gradient solver timeouts.
- Quantifying Estimator Variance: Running 5-fold cross-validation (cv=5) yielded fold accuracies of [0.7877, 0.7809, 0.7865, 0.7753, 0.8034]. This produced:
    - Mean CV Score: 78.68% o
      Standard Deviation (σ): 0.0094

The sub-1% dispersion confirmed that our initial 80% split was not an overfitted anomaly, but a statistically stable estimate of real-world generalization.

3. Connections & Methodological Evolution

This lab transformed how I view model deployment. In regression (e.g., Lab 02), we optimized continuous error functions like MSE. In classification, optimization is governed by the operational cost of error:

- High Precision: Minimizes Type I errors (False Positives) via elevated decision thresholds ($p > 0.7$)—critical for fraud detection where freezing an innocent account disrupts business.
- High Recall: Minimizes Type II errors (False Negatives) via lowered decision thresholds ($p > 0.3$)—critical for cancer diagnostics where a missed case is fatal.

I will no longer rely on single train-test splits or aggregate accuracy. Reliable evaluation requires the full diagnostic trio: stratified cross-validation for stability (σ), confusion matrices for error asymmetry, and threshold tuning aligned with the asymmetric cost of failure.

Module 07 Lab – Reflective Journal
ITAI-1371: Introduction to Machine Learning | Model Evaluation

Hamza Ajmal

Understanding why evaluation matters

This lab helped me understand that building a model is only one part of a machine-learning workflow. After a model makes predictions, I still need to check how well those predictions match the true labels. A single score can be useful, but it doe s not explain every type of mistake. The evaluation method should match the problem and should help answer a practical question about the model's performance.

Reading a confusion matrix

The confusion matrix made the model's correct and incorrect predictions easier to separate. For a binary classification problem, it shows true positives, true negatives, false positives, and false negatives. I learned that these error types are not interchangeable: a false positive is different from a false negative, and which one matters more depends on the application. Looking at the matrix alongside accuracy gives a more detailed view than relying on accuracy alone.

Precision, recall, and F1-score

The classification report introduced three related but different measures. Precision asks how often positive predictions are correct. Recall asks how many of the actual positive cases the model finds. F1-score combines precision and recall into one measure. Thinking about these measures together helped me see why a model can have reasonable accuracy while still missing important cases or producing too many false alarms. The metric I focus on should depend on the costs of different errors.

Why cross-validation adds information

The five-fold cross-validation task showed that evaluation can depend on which examples happen to be in one train/test split. Cross-validation repeats the evaluation across different folds and summarizes the scores, giving a more stable view of model performance. The fold scores can vary, so I should look at both the average and the spread rather than treating one result as a guarantee. Cross-validation does not remove every limitation, but it provides more evidence than one split alone.

Workflow complexity and what I would improve

This workflow also reminded me that data preparation, model training, and evaluation are connected. If the labels or split are wrong, the evaluation is not trustworthy even if the code runs. I should run cells in order, inspect the outputs, and investigate unexpected results instead of

assuming that successful execution means a model is good. In a future project, I would compare several appropriate metrics, inspect the confusion matrix, and consider whether the class distribution is imbalanced before deciding which model is preferable.

Personal observation from my run

Add one or two sentences here describing a real observation from your own run—for example, which confusion-matrix error surprised you, how your five fold scores differed, or what you had to troubleshoot. Keep only details that match what you personally observed.

Module 07 Lab – Reflective Journal

ITAI-1371: Introduction to Machine Learning | Model Evaluation

Jonathon Wilturner

Understanding why evaluation matters

This lab helped me understand that building a model is only one part of a machine -learning workflow. After a model makes predictions, I still need to check how well those predictions match the true labels. A single score can be useful, but it doe s not explain every type of mistake. The evaluation method should match the problem and should help answer a practical question about the model's performance.

Reading a confusion matrix

The confusion matrix made the model's correct and incorrect predictions easier to separate. For a binary classification problem, it shows true positives, true negatives, false positives, and false negatives. I learned that these error types are not interchangeable: a false positive is different from a false negative, and which one matters more depends on the application. Looking at the matrix alongside accuracy gives a more detailed view than relying on accuracy alone.

Precision, recall, and F1-score

The classification report introduced three related but different measures. Precision asks how often positive predictions are correct. Recall asks how many of the actual positive cases the model finds. F1-score combines precision and recall into one measure. Thinking about these measures together helped me see why a model can have reasonable accuracy while still missing important cases or producing too many false alarms. The metric I focus on should depend on the costs of different errors.

Why cross-validation adds information

The five-fold cross-validation task showed that evaluation can depend on which examples happen to be in one train/test split. Cross-validation repeats the evaluation across different folds and summarizes the scores, giving a more stable view of model performance. The fold scores can vary, so I should look at both the average and the spread rather than treating one result as a guarantee. Cross-validation does not remove every limitation, but it provides more evidence than one split alone.

Workflow complexity and what I would improve

This workflow also reminded me that data preparation, model training, and evaluation are connected. If the labels or split are wrong, the evaluation is not trustworthy even if the code runs. I should run cells in order, inspect the outputs, and investigate unexpected results instead of assuming that successful execution means a model is good. In a future project, I would compare several appropriate metrics, inspect the confusion matrix, and consider whether the class distribution is imbalanced before deciding which model is preferable.

Personal observation from my run

Add one or two sentences here describing a real observation from your own run—for example, which confusion-matrix error surprised you, how your five fold scores differed, or what you had to troubleshoot. Keep only details that match what you personally observed.